1. Ce tip de fișier este generat dintr-un schematic în Fusion 360?
  - ( ) .cad
  - ( ) .txt
  - (X) .sch
  - ( ) .doc

2. Când adăugați o nouă piesă în schematicul din Fusion 360, care este primul pas?
  - ( ) Selectați "New PCB"
  - (X) Click pe "New Schematic"
  - ( ) Salvare schematic
  - ( ) Generați un report

3. Care este procedura corectă pentru a plasa o piesă în schematicul Fusion 360?
  - (X) Dublu click pe piesa dorită și apăsați Enter
  - ( ) Tastați numele piesei și apăsați Ctrl+V
  - ( ) Drag and drop din lista de piese
  - ( ) Scrieți codul piesei într-un fișier .sch

4. Ce permite Fusion 360 să verificați înainte de fabricarea plăcii fizice?
  - ( ) Dimensiunea textului pe placa de circuit
  - ( ) Culoarea plăcii
  - (X) Erori de conexiune în schematic și board
  - ( ) Costul componentelor

5. În Fusion 360, ce reprezintă un "Board" generat din schematic?
  - ( ) O listă a componentelor
  - (X) Layout-ul fizic al plăcii de circuit
  - ( ) Un diagramă a conexiunilor logice
  - ( ) O imagine a schematicului

6. Ce reprezintă un "Symbol" în contextul creării unei piese în Fusion 360?
  - ( ) Dimensiunea fizică a piesei
  - ( ) Un test pentru verificarea funcționalității piesei
  - (X) Reprezentarea grafică a piesei în schematic
  - ( ) O descriere textuală a piesei

7. Ce trebuie să conțină un "Device" în Fusion 360?
  - (X) Un Symbol, un Package, și conexiunile dintre acestea
  - ( ) Doar un Package
  - ( ) Doar un Symbol
  - ( ) Nicio componentă, este doar o descriere

8. Unde este plasat un "Package" în contextul Fusion 360?
  - ( ) În schematic

- (X) În layout-ul board-ului
- ( ) În documentația proiectului
- ( ) Pe placa fizică a circuitului

9. Cum poate fi creat un nou "Package" pentru o piesă în Fusion 360?
- ( ) Prin desenarea directă pe placa de circuit
- (X) Prin definirea dimensiunilor fizice și a amplasamentului pinilor
- ( ) Prin scrierea codului piesei într-un editor de text
- ( ) Nu este necesar să creați un package nou

10. Ce opțiune permite adăugarea de text la un Symbol în Fusion 360?
- ( ) Line
- ( ) Pin
- ( ) Rotate
- (X) Text

11. Cum influențează alegerea layer-ului în procesul de creare a unui Symbol în Fusion 360 și care este layer-ul implicit utilizat pentru definirea simbolurilor?
- ( ) Layer-ul nu influențează crearea Symbol-ului; este doar o metodă de organizare vizuală.
- (X) Layer-ul 94 Symbols este utilizat implicit și este crucial pentru a asigura vizibilitatea și compatibilitatea simbolului în schematic.
- ( ) Orice layer poate fi folosit fără impact asupra funcționalității sau vizibilității simbolului.
- ( ) Layer-ul ales determină culorile și stilurile liniilor în simbol, afectând astfel doar estetica.

12. Care este rolul funcției "Connect" în procesul de legare a unui Symbol la un Package în crearea unui Device în Fusion 360, și ce alternative oferă interfața pentru asignarea multiplelor pad-uri la un pin?
- ( ) Funcția "Connect" este folosită doar pentru a redenumi pinii, iar asignarea se face automat.
- ( ) Funcția "Connect" schimbă fizic locația pinilor și pad-urilor pe board.
- (X) Funcția "Connect" permite asignarea manuală a pinilor la pad-uri și, prin funcția "Append", se pot asigna multiple pad-uri la un singur pin.
- ( ) Funcția "Connect" este folosită exclusiv pentru testarea conexiunilor și nu afectează design-ul.

13. În contextul adăugării de noi componente în Fusion 360, care sunt implicațiile alegerii incorecte a dimensiunilor pad-urilor în procesul de design al footprint-ului?
- ( ) Implicații estetice, modificând doar aspectul vizual al board-ului.
- (X) Pot apărea probleme de conectivitate și de montare, afectând fiabilitatea conexiunilor electrice și mecanice.
- ( ) Nu există implicații semnificative, deoarece Fusion 360 ajustează automat dimensiunile pad-urilor.
- ( ) Erori în generarea listei de materiale, afectând doar documentația.

14. Ce strategii pot fi aplicate pentru gestionarea eficientă a bibliotecilor în Fusion 360 când se lucrează cu componente multiple și diverse?
- ( ) Utilizarea unei singure biblioteci universale pentru toate proiectele.
- (X) Organizarea componentelor în biblioteci separate pe baza categoriilor funcționale și actualizarea regulată a acestora.
- ( ) Limitarea numărului de componente per bibliotecă pentru a minimiza dimensiunea fișierelor.
- ( ) Evitarea creării de biblioteci custom și utilizarea exclusivă a bibliotecilor predefinite.

15. Cum poate influența optimizarea layout-ului unui PCB creat în Fusion 360 performanțele circuitului și care sunt practicile recomandate pentru plasarea componentelor și traseelor?
- (X) Optimizarea layout-ului poate reduce interferențele electromagnetice și poate îmbunătăți eficiența termică, recomandându-se minimizarea lungimii traseelor de putere și plasarea strategică a condensatoarelor de decuplare.
- ( ) Layout-ul optimizat este pur estetic și nu afectează performanțele circuitului.
- ( ) Prin optimizarea layout-ului se economisește spațiu pe board, afectând doar costurile de producție.
- ( ) Recomandarea principală este maximizarea distanței între trasee pentru a facilita intervențiile manuale.

16. Ce metodă se folosește pentru a optimiza manual rutarea traseelor în Fusion 360 și ce tipuri de rutare pot fi selectate din meniul de rutare manuală?
- ( ) Se folosește exclusiv autorutarea, fără opțiuni manuale disponibile.
- (X) Rutarea manuală permite selectarea stilurilor de rutare precum Trace Bend și Right Angle pentru optimizarea traseului.
- ( ) Rutarea se face automat, iar utilizatorul nu poate alege stiluri diferite.
- ( ) Manual se pot adăuga doar conexiuni temporare, fiind necesară confirmarea de către un administrator.

17. Cum poate fi implementat un plan de masă eficient în Fusion 360 și care sunt pașii necesari pentru crearea acestuia?
- (X) Prin utilizarea funcției Polygon Pour, delimitând zona dorită și redenumind poligonul cu numele GND pentru a indica conexiunea la masă.
- ( ) Planul de masă este creat automat de Fusion 360, fără intervenție manuală.
- ( ) Prin desenarea manuală a fiecărui traseu care conectează la masă, fără a folosi funcții automate.
- ( ) Este suficient să se plaseze o linie de conexiune la masă în jurul perimetrului board-ului.

18. Care sunt beneficiile utilizării planului de masă în design-ul unui PCB și cum influențează acesta performanța circuitului?
- (X) Reduce zgomotul și interferențele, facilitând accesul componentelor la masă și îmbunătățind integritatea semnalului.
- ( ) Principala funcție este estetică, îmbunătățind aspectul vizual al PCB-ului.
- ( ) Beneficiile sunt exclusiv în termeni de cost, reducând cantitatea de cupru necesară.

- ( ) Planul de masă este utilizat doar pentru a identifica erori de rutare, fără impact asupra performanței.

19. În ce mod poate afecta incorecta configurare a planului de masă în Fusion 360 funcționalitatea generală a unui PCB?
- ( ) Crește capacitățile de stocare de date ale PCB-ului.
- (X) Poate duce la bucle de masă și probleme EMC, degradând performanța circuitului.
- ( ) Afectează doar timpul de încărcare al proiectului în software.
- ( ) Modifică doar dimensiunile fizice ale PCB-ului, fără efecte asupra funcționalității.

20. Cum poate fi utilizată funcția "Hide All Polygon Pour Fills" în gestionarea vizualizării planurilor de masă în Fusion 360 și care sunt avantajele acestei funcții?
- (X) Permite ascunderea temporară a umplerilor poligonale pentru a facilita editarea altor elemente ale PCB-ului fără distragere vizuală.
- ( ) Această funcție schimbă culoarea planurilor pentru a distinge între diferitele nivele de tensiune.
- ( ) Reduce consumul de memorie al aplicației prin ascunderea elementelor grafice.
- ( ) Este folosită doar la exportul designului pentru producție, fără utilitate în procesul de design.

21. Ce comandă Git permite verificarea stării modificărilor locale în raport cu repository-ul remote?
- ( ) `git commit`
- ( ) `git push`
- (X) `git status`
- ( ) `git merge`

22. Care dintre următoarele afirmații descrie corect conceptul de "fork" în contextul GitHub?
- ( ) Un fork este o copie privată a unui repository, accesibilă doar administratorului original.
- ( ) Un fork reprezintă clonarea unui repository pentru vizualizare temporară, fără permisiunea de a modifica.
- (X) Un fork este o copie a unui repository care permite utilizatorului să propună modificări fără a afecta repository-ul original.
- ( ) Un fork este o comandă Git pentru îmbinarea a două branch-uri.

23. Cum se poate combina un branch de dezvoltare cu branch-ul principal în Git, și ce comandă se utilizează?
- ( ) Se utilizează comanda `git combine` pentru a unifica branch-urile.
- ( ) Se utilizează comanda `git merge` pentru a adăuga commit-uri din branch-ul de dezvoltare în branch-ul principal.
- (X) Se utilizează comanda `git merge` în branch-ul principal pentru a încorpora schimbările din branch-ul de dezvoltare.
- ( ) Se utilizează comanda `git push` pentru a trimite schimbările direct în branch-ul principal.

24. Care este scopul utilizării `.gitignore` într-un proiect Git?
- ( ) Pentru a ignora mesajele de eroare generate de Git.
- (X) Pentru a specifica fișierele și directoarele pe care Git ar trebui să le ignore la commit-uri.
- ( ) Pentru a exclude utilizatori specifici de la a face commit-uri în proiect.
- ( ) Pentru a bloca descărcarea proiectului de pe GitHub.

25. Ce reprezintă termenii "upstream" și "origin" în contextul Git, și cum sunt folosiți în colaborarea pe GitHub?
- ( ) "Upstream" se referă la branch-ul principal de unde se clonează proiectul, iar "origin" este branch-ul de dezvoltare.
- (X) "Origin" este repository-ul default remote de unde a fost clonat proiectul, iar "upstream" este folosit pentru a referi repository-ul original în cazul unui fork.
- ( ) "Upstream" și "origin" sunt comenzi pentru sincronizarea rapidă a repository-urilor.
- ( ) "Origin" este versiunea locală a repository-ului, iar "upstream" este branch-ul de backup automat.

26. Ce afirmatie descrie corect utilizarea markerelor în pytest?
- ( ) Markerele în pytest sunt folosite exclusiv pentru a marca testele care ar trebui să ruleze într-un mediu de producție.
- ( ) Markerele nu sunt suportate în pytest; ele sunt o caracteristică exclusivă a unittest din Python.
- (X) Markerele în pytest sunt utilizate pentru a adăuga metadate la testele tale, cum ar fi skip sau xfail, care controlează cum sunt rulate testele.
- ( ) Markerele sunt folosite pentru a șterge rezultatele testelor pentru a păstra curățenia log-urilor.

27. Cum se folosește fixture în pytest pentru a pregăti datele necesare unui test?
- ( ) Fixture-urile sunt folosite pentru a configura baze de date externe înainte de fiecare test.
- (X) O fixture este o funcție decorată cu `@pytest.fixture`, care poate fi folosită pentru a seta un mediu de test (ex: configurarea unei baze de date test) și este injectată în teste prin parametrii funcției de test.
- ( ) Fixture-urile sunt metode speciale care rulează după fiecare funcție de test pentru a curăța modificările.
- ( ) În pytest, fixture-urile sunt strict legate de modulul în care sunt definite și nu pot fi partajate între mai multe module de test.

28. Care este scopul folosirii `pytest.raises` în cadrul unui test?
- ( ) Pentru a verifica dacă un test nu generează nicio excepție.
- ( ) `pytest.raises` este folosit pentru a opri executarea tuturor testelor la prima excepție întâlnită.
- (X) Pentru a verifica dacă un anumit bloc de cod aruncă o excepție așteptată în timpul execuției testului.
- ( ) Este utilizat exclusiv pentru a captura și a loga excepțiile care nu sunt legate de testele curente.

29. Cum se poate implementa parametrizarea unui test în pytest pentru a rula același test cu diferite seturi de date?
- ( ) Prin duplicarea codului testului pentru fiecare set de date.
- ( ) Parametrizarea nu este suportată în pytest; fiecare set de date necesită un test separat.
- (X) Utilizând decoratorul `@pytest.mark.parametrize`, care permite specificarea mai multor seturi de argumente pentru care testul va fi rulat repetat.
- ( ) Prin scrierea unei funcții speciale care generează seturi de date în timpul rulării testelor.

30. Ce este o "test suite" în contextul pytest și cum poate fi configurată pentru a rula multiple test cases?
- ( ) O "test suite" este un set de comenzi care trebuie executate manual înainte de fiecare sesiune de testare.
- (X) O "test suite" în pytest poate fi configurată utilizând fișiere de configurare (`pytest.ini` sau `conftest.py`) pentru a include și a exclude teste specifice bazate pe markere, directoare, etc.
- ( ) "Test suite" se referă la mediul fizic în care se rulază testele și este configurat prin modificarea setărilor serverului.
- ( ) În pytest, fiecare fișier de test este considerat automat o "test suite", și nu necesită configurări suplimentare pentru a include mai multe teste.

31. Cum poate fi folosită funcționalitatea de "mocking" în pytest pentru a testa componente care depind de servicii externe?
- ( ) Prin scrierea unui script separat care imită răspunsul serviciului.
- (X) Utilizând modulul `pytest-mock` sau `unittest.mock` pentru a înlocui obiectele și metodele externe cu variante controlate care returnează valori predeterminate.
- ( ) Mocking nu este recomandat în pytest deoarece poate duce la rezultate de test nerealiste.
- ( ) Prin configurarea serverelor de test care emulează serviciile externe.

32. Care este scopul utilizării `pytest-cov` într-un proiect pytest și cum este utilizat efectiv pentru a îmbunătăți calitatea codului?
- ( ) `pytest-cov` este utilizat pentru a mări viteza de rulare a testelor prin reducerea numărului de teste executate.
- (X) `pytest-cov` este un plugin pentru pytest care generează rapoarte de acoperire a codului, ajutând la identificarea părților de cod neacoperite de teste.
- ( ) Este folosit exclusiv pentru proiectele mari, complexe, și nu oferă beneficii pentru proiectele mai mici.
- ( ) `pytest-cov` restricționează execuția testelor la modulele specificate, ignorând restul proiectului.

33. Ce strategie se utilizează în pytest pentru a gestiona testele care trebuie să ruleze în condiții specifice de mediu sau configurare?
- ( ) Toate testele trebuie să fie independente de mediu, astfel această situație nu se aplică.

- (X) Utilizarea markerelor custom, cum ar fi `@pytest.mark.environment('production')`, pentru a eticheta și selecta testele în funcție de mediul specific.
- ( ) Modificarea codului de bază pentru a include condiții de mediu direct în logica testelor.
- ( ) Utilizarea variabilelor de mediu pentru a configura dinamic comportamentul testelor.

34. Cum poate fi testată o eroare de timeout într-un sistem care face apeluri de rețea, folosind pytest?
- (X) Simulând un timeout folosind mocking pentru a înlocui funcția de rețea cu una care ridică o excepție de timeout.
- ( ) Prin deconectarea fizică a serverului în timpul testului pentru a genera un timeout natural.
- ( ) Timeout-urile nu pot fi testate în pytest deoarece sunt dependente de infrastructura de rețea.
- ( ) Folosind o buclă infinită în test pentru a forța condițiile de timeout.

35. În contextul pytest, cum poate fi utilizată funcția `pytest_generate_tests` pentru a controla generarea parametrilor de test?
- ( ) Această funcție este utilizată pentru a genera teste automat pe baza codului sursă.
- ( ) `pytest_generate_tests` este folosită pentru a crea automat date de test bazate pe algoritmi de inteligență artificială.
- (X) `pytest_generate_tests` permite definirea dinamică a parametrilor de test prin hook-uri, facilitând teste complexe parametrizate personalizat.
- ( ) Este folosită doar pentru a reordona testele, fără a influența parametrii acestora.

36. Cum poate fi utilizată funcția `breakpoint()` în Python pentru debugging în timpul rulării unui test?
- ( ) Pentru a opri execuția programului la fiecare test.
- (X) Pentru a insera puncte de oprire în cod, permițând inspectarea stării programului în timpul execuției testelor.
- ( ) Breakpoint-ul este utilizat doar în producție pentru a semnala erori critice.
- ( ) Funcția `breakpoint()` nu este disponibilă în Python, se folosește `sys.exit()`.

37. Ce rol are utilizarea aserțiunilor în scripturile de testare și care este efectul lor asupra ciclului de dezvoltare software?
- ( ) Aserțiunile sunt folosite doar pentru a verifica stilul codului, nu comportamentul.
- ( ) Aserțiunile încetinesc testele și sunt evitate în testarea modernă.
- (X) Aserțiunile verifică dacă condițiile esențiale sunt îndeplinite în cod, ajutând la identificarea rapidă a defectelor.
- ( ) Aserțiunile sunt convertite automat în teste de performanță.

38. Care este utilizarea `@pytest.mark.skipif` în cadrul suitei de teste pytest?
- ( ) Pentru a repeta un test de mai multe ori bazat pe o condiție.
- (X) Pentru a sări peste executarea unui test dacă o condiție specificată este adevărată.
- ( ) Se folosește pentru a marca testele ca fiind irelevante și a le elimina din rapoartele finale.
- ( ) Aceasta marchează testele ca fiind critice și le execută în primul rând.

39. Cum se poate realiza testarea performanței unui cod în Python, și ce unelte sunt recomandate pentru acest scop?
- (X) Folosind modulele `timeit` sau `cProfile` pentru a evalua performanța execuției codului.
- ( ) Performanța este evaluată manual, cronometrând timpul de execuție cu un ceas.
- ( ) Testarea performanței este gestionată exclusiv de IDE-uri și nu prin scripturi Python.
- ( ) Utilizând `pytest-timeout` pentru a verifica cât timp durează fiecare test.

40. În contextul testării software, ce este un "test de regresie" și când ar trebui acesta implementat în ciclul de dezvoltare?
- ( ) Testele de regresie verifică dacă software-ul poate rula pe sisteme mai vechi.
- ( ) Este un test care se efectuează doar după ce produsul a fost livrat.
- (X) Testele de regresie verifică dacă modificările recente nu au afectat funcționalitățile existente anterior.
- ( ) Se aplică doar în testarea bazată pe model, pentru a verifica aderența la specificații.

41. Ce este integrarea continuă (CI) și cum contribuie la procesul de dezvoltare software?
- ( ) CI este o practică care permite dezvoltatorilor să modifice interfața utilizatorului fără a afecta codul de bază.
- ( ) CI se referă la actualizarea continuă a documentației software pentru a reflecta schimbările de cod.
- (X) CI este o practică de dezvoltare software în care membrii echipei integrează lucrul lor frecvent, de obicei fiecare integrare fiind verificată printr-un build automat pentru a detecta erori cât mai curând posibil.
- ( ) CI este procesul prin care codul este evaluat de un comitet de revizuire înainte de a fi împins în producție.

42. Cum poate fi folosită Docker în contextul CI pentru testarea aplicațiilor?
- (X) Docker poate fi utilizat pentru a crea medii de testare izolate și reproduceabile, asigurând că testele rulează într-un mediu controlat și consistent.
- ( ) Docker este folosit exclusiv pentru stocarea imaginilor de backup ale codului înainte de integrare.
- ( ) Docker este utilizat pentru a crește viteza de download a dependințelor necesare în timpul build-ului.
- ( ) Docker optimizează performanța aplicațiilor în timpul testelor, prin alocarea dinamică a resurselor.

43. Care este scopul utilizării unui server dedicat de CI, cum ar fi Jenkins, în dezvoltarea software?
- ( ) Pentru a facilita comunicarea între membrii echipei de dezvoltare prin chat și e-mail integrat.
- (X) Pentru a automatiza procesul de build și test, rulând teste pe modificările de cod înainte ca acestea să fie integrate în branch-ul principal.
- ( ) Jenkins este folosit exclusiv pentru monitorizarea performanței aplicației în producție.

- ( ) Jenkins servește ca o bază de date pentru stocarea versiunilor anterioare ale software-ului.

44. Ce reprezintă "testele de fum" (smoke testing) în cadrul unui proces de CI și care este importanța lor?
- (X) Testele de fum verifică funcționalitățile de bază ale unei aplicații pentru a asigura că modificările majore nu strică aplicația în moduri evidente.
- ( ) Sunt teste care se efectuează doar în medii de producție pentru a verifica impactul pe utilizatori reali.
- ( ) Testele de fum sunt efectuate de utilizatori externi pentru a evalua interfața utilizatorului.
- ( ) Aceste teste sunt utilizate pentru a verifica compatibilitatea aplicației pe diferite dispozitive mobile.

45. Cum se utilizează testarea bazată pe performanță în cadrul ciclurilor de CI/CD pentru a îmbunătăți calitatea software?
- ( ) Se aplică doar la aplicații de înaltă performanță, cum ar fi jocurile sau aplicațiile de streaming video.
- ( ) Testarea performanței este neglijată în CI/CD deoarece focusul este pe integrarea caracteristicilor noi.
- (X) Testarea performanței este integrată în ciclurile de CI/CD pentru a evalua cum schimbările de cod afectează timpul de răspuns și eficiența resurselor, asigurând că performanța nu se degradează odată cu adăugarea de noi funcționalități.
- ( ) Se folosește exclusiv pentru testarea serverelor de bază de date în medii cloud.

46. Ce metodă poate fi utilizată pentru a optimiza performanța unei aplicații web în faza de pre-deployment?
- (X) Profilarea aplicației pentru a identifica și îmbunătăți funcțiile care consumă cele mai multe resurse.
- ( ) Schimbarea întregului stack tehnologic bazat pe ultimele trenduri în dezvoltare.
- ( ) Limitarea accesului la aplicație pentru a reduce încărcarea serverelor.
- ( ) Utilizarea exclusivă a cache-ului static pentru conținutul dinamic.

47. Cum poate fi implementată securitatea la nivelul aplicației într-un mediu cloud?
- ( ) Prin dezactivarea tuturor serviciilor neutilizate, chiar și cele critice pentru funcționare.
- (X) Utilizarea grupurilor de securitate, listelor de control acces și criptarea datelor atât în repaus cât și în tranzit.
- ( ) Implementarea unui singur model de securitate care nu este configurabil.
- ( ) Limitarea accesului la aplicație doar pentru utilizatorii din rețeaua internă a companiei.

48. Care este rolul automatizării în procesul de deployment continuu (CD)?
- ( ) Automatizarea este folosită doar pentru testarea aplicațiilor înainte de deployment.
- ( ) CD se concentrează exclusiv pe manualizarea procesului de deployment pentru a reduce erorile.

- (X) Automatizarea în CD permite rularea scripturilor de deployment în mod repetabil și consistent, reducând erorile umane și accelerând lansările.
- ( ) CD automatizează procesul de dezvoltare, nu și pe cel de deployment.

49. Cum pot fi utilizate containerele, cum ar fi Docker, pentru a îmbunătăți procesul de deployment?
- ( ) Containerele sunt folosite pentru a crește dimensiunea aplicațiilor și a testa limita de stocare.
- ( ) Docker este relevant doar pentru dezvoltarea locală și nu are aplicabilitate în producție.
- (X) Docker poate asigura consistență între medii prin izolarea dependențelor aplicației, facilitând astfel un deployment predictibil și scalabil.
- ( ) Utilizarea containerelor încetinește deployment-ul deoarece complică gestionarea resurselor.

50. Ce strategii pot fi aplicate pentru a asigura scalabilitatea unei aplicații în timpul pieților de vârf?
- ( ) Scalabilitatea este gestionată manual prin adăugarea fizică a resurselor hardware în momentul necesar.
- ( ) Reducerea funcționalităților aplicației pentru a diminua sarcina pe servere.
- (X) Implementarea scalabilității orizontale prin adăugarea de instanțe suplimentare ale aplicației în funcție de cerere și utilizarea balansatorilor de sarcină.
- ( ) Folosirea unui singur server puternic pentru a gestiona toate cererile, indiferent de volum.

1. Cum verificăm existența unei erori de conexiune într-un circuit proiectat cu Fusion 360?
- ( ) Verificând manual fiecare conexiune.
- (X) Utilizând funcția de Electrical Rule Check (ERC).
- ( ) Prin inspecție vizuală după imprimarea PCB-ului.
- ( ) Folosind un multimetru pe placa fizică.

2. Care este scopul principal al utilizării unui Ground Plane într-un design PCB?
- ( ) Pentru a estetiza designul PCB-ului.
- ( ) Pentru a adăuga greutate și durabilitate plăcii.
- (X) Pentru a reduce zgomotul și a îmbunătăți performanța electrică a circuitului.
- ( ) Pentru a separa componentele electrice de PCB.

3. Ce reprezintă un 'Via' în contextul unui PCB proiectat cu Fusion 360?
- (X) O conexiune electrică între diferite straturi ale PCB-ului.
- ( ) O greșeală de proiectare care trebuie evitată.
- ( ) O componentă care conectează PCB-ul la sursa de alimentare.
- ( ) Un tip de componentă electronică montată pe suprafață.

4. În ce situație este necesar să utilizăm funcția `git rebase` în loc de `git merge`?
- ( ) Când vrem să ștergem un commit din istoric.

- (X) Când dorim să integrăm schimbări recente din branch-ul principal fără a crea un commit de merge.
- ( ) Când trebuie să combinăm două repository-uri diferite.
- ( ) Când serverul remote nu acceptă push-uri.

5. Cum pot fi detectate regresiile de performanță într-un proiect software folosind testele automate?
- ( ) Verificând manual timpul de răspuns al aplicației după fiecare commit.
- ( ) Prin folosirea unui software de monitorizare a utilizatorilor.
- (X) Integrând teste de performanță în pipeline-ul de CI/CD și comparând rezultatele cu benchmark-uri anterioare.
- ( ) Așteptând feedback de la utilizatori.

6. Ce strategie de testare asigură că modificările codului respectă cerințele inițiale fără a introduce defecte noi?
- ( ) Testarea continuă a performanței.
- ( ) Testarea funcțională manuală.
- (X) Testarea de regresie automată.
- ( ) Refacerea testelor de acceptanță de către client.

7. Cum influențează practicile de Continuous Integration (CI) calitatea software-ului dezvoltat?
- ( ) Reducând costurile de dezvoltare prin eliminarea testării.
- ( ) Creșterea numărului de defecte prin testare excesivă.
- (X) Îmbunătățirea calității prin detectarea timpurie și corectarea defectelor.
- ( ) Ignorarea defectelor până în faza de producție.

8. Care este beneficiul principal al utilizării containerizării în testarea software?
- ( ) Reducerea costurilor de licențiere a software-ului.
- (X) Crearea de medii de testare consistente și izolate.
- ( ) Accelerarea procesului de dezvoltare prin eliminarea testării.
- ( ) Limitarea accesului la resursele de test.

9. Ce metodologie de testare implică evaluarea unui produs software fără a avea acces la structura internă a codului sursă?
- ( ) Testarea white-box.
- (X) Testarea black-box.
- ( ) Testarea grey-box.
- ( ) Testarea green-box.

10. În contextul testării software, ce înseamnă un 'mock object'?
- ( ) Un obiect real utilizat pentru a simula comportamentul utilizatorului.
- (X) Un obiect fictiv configurat în testele automate pentru a simula comportamentul dependențelor.
- ( ) O componentă software defectă utilizată intenționat pentru a testa robustețea aplicației.

- ( ) Un instrument de testare care detectează automat defectele.

11. Cum poate fi testată securitatea unei aplicații web în faza de dezvoltare?
- ( ) Prin manualizarea proceselor de securitate și evitarea testelor automate.
- (X) Utilizând teste automate de penetrare și scanări de vulnerabilități.
- ( ) Limitând testele de securitate doar la componente non-critice.
- ( ) Folosind doar revizuirea codului manual pentru securitate.

12. Care este avantajul principal al utilizării testelor unitare în dezvoltarea software?
- ( ) Reducerea nevoii de documentație tehnică.
- (X) Izolarea și identificarea rapidă a defectelor la nivel de componentă.
- ( ) Eliminarea completă a defectelor software.
- ( ) Simplificarea interfeței utilizator.

13. Ce rol joacă 'Continuous Deployment' în ciclul de dezvoltare a software-ului?
- ( ) Este folosit exclusiv pentru proiecte mici, cu cicluri scurte de dezvoltare.
- ( ) Elimină necesitatea testării software.
- (X) Asigură lansarea automată a software-ului în producție după fiecare schimbare verificată.
- ( ) Reduce frecvența actualizărilor software.

14. Cum contribuie folosirea unui repository de artefacte, cum ar fi JFrog Artifactory, la procesul de CI/CD?
- ( ) Prin limitarea accesului la artefacte la membrii echipei de dezvoltare.
- (X) Oferind un loc centralizat și securizat pentru stocarea și gestionarea dependențelor și a binarelor.
- ( ) Prin eliminarea necesității de a folosi un sistem de versionare.
- ( ) Facilitând exclusiv partajarea artefactelor între diferite echipe.

15. Care este scopul utilizării unui fuzzer în testarea software?
- ( ) Pentru a reduce timpul necesar scrierii testelor manuale.
- ( ) Pentru a verifica acuratețea documentației software.
- (X) Pentru a genera inputuri neașteptate sau aleatorii pentru a descoperi defecte și vulnerabilități.
- ( ) Pentru a asigura conformitatea codului cu standardele de codificare.

16. În ce moduri poate fi îmbunătățită eficiența executării testelor automate într-un mediu de integrare continuă?
- ( ) Prin executarea testelor doar în afara orelor de lucru.
- (X) Utilizând paralelizarea testelor și gestionarea inteligentă a resurselor.
- ( ) Limitând testele la cele care eșuează frecvent.
- ( ) Ignorând testele pentru funcționalități mai vechi.

17. Care este avantajul implementării testelor de integrare față de testele unitare?

- ( ) Testele de integrare sunt mai rapide și mai ușor de scris.
- (X) Verifică modul în care componentele software interacționează între ele, detectând probleme care nu sunt evidente la nivel unitar.
- ( ) Pot înlocui complet testele unitare.
- ( ) Sunt mai puțin costisitoare decât testele unitare.

18. Cum poate fi utilizată instrumentația dinamică în monitorizarea performanței aplicațiilor?
- ( ) Prin modificarea codului sursă al aplicației pentru a include instrucțiuni de monitorizare.
- (X) Introducând sondaje (probes) la runtime care colectează date despre comportamentul aplicației fără a afecta sistemul de producție.
- ( ) Utilizând exclusiv simulări offline pentru evaluarea performanței.
- ( ) Prin restrângerea monitorizării doar la medii de testare.

19. Ce reprezintă testarea bazată pe proprietăți și cum se aplică în dezvoltarea software?
- ( ) Este o metodă care testează doar proprietățile de bază ale sistemului, cum ar fi interfața grafică.
- (X) O tehnică în care testele verifică dacă proprietățile logic definite ale software-ului sunt respectate pentru diverse intrări.
- ( ) Se referă la testarea manuală a proprietăților fișierelor de configurare.
- ( ) Este utilizată exclusiv pentru testarea sistemelor de bază de date.

20. Cum influențează microserviciile strategia de testare într-un proiect software?
- ( ) Prin eliminarea necesității de a testa interfețele dintre componentele sistemului.
- (X) Necesită o abordare complexă care include testarea individuală a serviciilor și testarea integrării lor.
- ( ) Microserviciile reduc complexitatea testării prin simplificarea arhitecturii.
- ( ) Permite utilizarea unor metode de testare mai puțin riguroase datorită izolării serviciilor.